

Projektvorschlag: Selfie/C* um Type Inference (auto) erweitern

Typsystem

Die Erweiterung bleibt bewusst innerhalb des Typsystems von C* (`uint64_t`, `uint64_t*`): `auto` leitet einen dieser beiden bestehenden Typen ab, ohne neue Typen oder implizite Konversionen einzuführen. Der interessante Inferenzfall liegt nicht bei primitiven Typen, sondern bei **Pointern**: `&`, `*` und Pointer-Arithmetik verändern den Typ — und genau diesen Typ liefert Selfies Ausdrucks-Compiler bereits als Rückgabewert zurück. Vergleiche liefern `uint64_t` mit Wert 0 oder 1, analog zu C:

Operation	Inferenz-Beispiel	Ergebnis
Integer-Literal	<code>auto n = 10;</code>	<code>uint64_t</code>
Adresse <code>&</code>	<code>auto p = &n;</code>	<code>uint64_t*</code>
String-Literal	<code>auto s = "Selfie";</code>	<code>uint64_t*</code>
Deref <code>*</code>	<code>auto x = *p;</code>	<code>uint64_t</code>
Ptr-Arithmetik	<code>auto q = p + 1;</code>	<code>uint64_t*</code>
Vergleich	<code>auto b = (n > 0);</code>	<code>uint64_t</code> (0/1)
Fehler	<code>p + q</code> (beide Ptr)	<i>Compilerfehler</i>

Beispielprogramm

```
uint64_t increment(uint64_t i) { return i + 1; }

uint64_t main() {
    auto n      = 10;           // uint64_t  -- Integer-Literal
    auto result = increment(n); // uint64_t  -- Rueckgabotyp der Funktion
    auto flag   = (result > 0); // uint64_t  -- Vergleich: 0 oder 1

    auto p      = &n;           // uint64_t* -- Adresse von n
    auto q      = p + 1;        // uint64_t* -- Pointer-Arithmetik
    auto x      = *p;           // uint64_t  -- Dereferenz

    auto s      = "Selfie";     // uint64_t* -- String-Literal (char*)
    // auto err = p + q;        // FEHLER   -- Pointer + Pointer ist undefiniert
    return x - n;               // 0 bei Erfolg
}
```

Architektur: weiterhin ein einziger Pass

Die Erweiterung verändert Selfies Single-Pass-Architektur *nicht*: Scanner → rekursiver Abstiegsparser + Code-Emitter → RISC-U. Es wird **kein** expliziter AST aufgebaut und **kein** eigener Typcheck-Pass eingeführt. Der Schlüssel ist, dass Selfies Ausdrucks-Compiler bereits lokal Typen verfolgt: Jede `compile_*`-Funktion gibt den Typ (`UINT64_T` oder `UINT64STAR_T`) des soeben erzeugten Werts zurück. Eine `auto`-Deklaration *liest diesen Typ lediglich* aus dem Rückgabewert von `compile_expression()` und trägt ihn in die Symboltabelle ein. Die Änderung betrifft drei Funktionen in `selfie.c`; der Code-Generator und das Typsystem bleiben unberührt.

Grenzen der Inferenz

Die Inferenz ist bewusst **lokal** gehalten: Der Typ einer `auto`-Variable wird ausschließlich aus dem direkten Initialisierer abgeleitet — nie aus späterem Verwendungskontext. Da im Single-Pass ein Bezeichner vor seiner Verwendung deklariert sein muss, sind zirkuläre und vorwärtsgerichtete Abhängigkeiten (`auto x = y; auto y = x;`) automatisch ein Fehler ("undeclared identifier"). Damit bleibt die Inferenz stets entscheidbar, auf Kosten von Ausdrucksstärke.